

7. bool match(char exp[]) {

int matchstat = 1; InitStack(&S);

ch = *exp++;

while (ch != '#' && matchstat) {

switch (ch) {

case '(':

case '[':

case '{': push(&S, ch); break;

case ')': if (!Pop(&S, &e) && e != '(')
matchstat = 0; break;

case ']': 类似上者

case '}': 类似上者 }

ch = *exp++; }

if (matchstat && StackEmpty(S)) return TRUE;

else return FALSE; }

优先级比较

8. bool preop(char a, char b) {

int a-level, b-level;

switch (a) { case '#': a-level = -1; break;

case '(': a-level = 0; break;

case '+':

case '-': a-level = 1; break;

case '*':

case '/': a-level = 2; break; } 同理 switch(b)

return a-level >= b-level;

}

转化成后缀表达式

void getsuffix(char exp[], char suffix[]) {

InitStack(S); push(S, '#');

char *p = exp; k = 0;

while (!StackEmpty(S)) {

ch = *p++;

if (!isoperator(ch)) { suffix[k++] = ch; continue; }

switch (ch) { case '(': push(S, ch); break;

case ')': while (pop(S, c) && c != '(')

suffix[k++] = c;

break;

default: while (GetTop(S, c) && preop(c, ch))

{ suffix[k++] = c; pop(S, c); }

if (ch != '#') push(S, ch);

break; }

suffix[k] = '\0'; DestroyStack(S); }

9. int calculate(char suffix[]) {

char *p = suffix; InitStack(S);

ch = *p++;

while (ch != '#') { if (!isoperator(ch)) push(S, ch);

else { pop(S, b); pop(S, a);

push(S, operate(a, ch, b)); }

ch = *p++; }

pop(S, e);

DestroyStack(S);

return (e); }

```

10. typedef struct LNode {
    ElemType data;
    struct LNode *next;
} LNode, *LinkQueueRear;

void InitQueue ( LinkQueueRear &rear) {
    rear->next = rear; }

void EnQueue ( LinkQueueRear &rear, ElemType e) {
    LNode *p= new LNode;
    p->data = e;
    p->next= rear->next
    rear->next=p;
    rear = p; }

bool DeQueue ( LinkQueueRear &rear, ElemType &e) {
    if (rear == rear->next) return FALSE;
    LNode *head= rear->next;
    LNode *p = head->next;
    e = p->data;
    if (p==rear) rear = rear->next;
    head->next= p->next;
    delete p; return TRUE; }

```

```

11. void EnQueue (SqQueue &Q, ElemType e) {
    if (Q.length == Q.maxsize) Increment (Q)
    Q.rear = (Q.rear+1) % Q.maxsize;
    Q.elem [Q.rear] = e;
    Q.length ++; }

bool DeQueue (SqQueue &Q, ElemType &e) {
    if (Q.length == 0) return FALSE;
    int front = (Q.rear - Q.length + 1 + Q.maxsize) % Q.maxsize;
    e = Q.elem [front];
    Q.length--; return TRUE; }

//注意: Q.length = Q.maxsize.

```